Caso semestral

Instrucciones - Caso

Docente / Estudiante

Descripción

- La red exige:
  - Login corporativo con Azure AD (IDaaS).
  - Frontend Angular con MSAL y autorización por rol (Admin, Operador, Cliente).
  - Backend Spring Boot con microservicios detrás de AWS API Gateway protegido con el JWT generado por Azure AD.
  - Despliegue en AWS EC2 con Docker/Docker-Compose.
  - Mensajería asíncrona con RabbitMQ (tareas/colas de trabajo) y streaming con Kafka (con Zookeeper) para analítica y auditoría en tiempo real.
- Alcance funcional minimo:

Módulo	Descripción	Actores	Reglas clave
Gestión de Pedidos	CRUD de pedidos, cambio de estado (CREADO -> ACEPTADO -> EN_PREPARACIÓN -> DESPACHADO -> ENTREGADO / CANCELADO)	Cliente, Operador	No se puede “despachar” sin “aceptar”
Catálogo	CRUD de productos y stock disponible	Admin	Stock decrece al aceptar pedido
Notificaciones	Email/push a cliente con estado de pedido	Operador, Cliente	Envío asíncrono (cola)
Reportería	Panel de KPIs: ventas por hora, lead time*, estados activos	Admin	Datos por streaming (Kafka) sin bloquear core
Auditoría	Timeline de eventos de negocio	Auditor	Solo lectura

*: lead time Es el tiempo transcurrido entre que un cliente crea el pedido y el momento en que ese pedido es entregado.

- Seguridad e identidad (IDaaS Azure + API Gateway)

  Azure AD (Entra External ID)

  - App Registration “Pedidos360”: clientId, redirectUri, authority = https://login.microsoftonline.com/<TENANT_ID>/

  MSAL Angular: proteger rutas y adjuntar Bearer <access_token> a cada request.

  AWS API Gateway (HTTP API): JWT Authorizer con:

  - issuer: https://login.microsoftonline.com/<TENANT_ID>/v2.0
  - audiences: api://<API_CLIENT_ID> Esto valida el JWT antes de enrutar a los microservicios.

  Spring Security:

  - Validar JWT llegado desde el frontend antes de procesar solicitud utilizando security: oauth2.resourceserver.jwt.issuer-uri
  - Validar que el rol que llega puede utilizar el endpoint que se está llamando.

Servicio	Dominio	DB	Responsabilidad	Exposición
ms-pedidos360-orders	Pedidos	Oracle	CRUD pedidos, estados, coordinación stock y notificación	/api/orders/*
ms-pedidos360-catalog-	Productos/Stock	Oracle	CRUD productos, stock y precios	/api/catalog/*
ms-pedidos360-notify	Notificaciones	sin DB	Procesa envío email/webpush **vía RabbitMQ**	no público (consumidor RabbitMQ)
ms-pedidos360-audit	Auditoría/Timeline	Oracle	Consume **Kafka** (más adelante) y persiste eventos	/api/audit/* (read-only)
ms-pedidos360-report	KPIs/Analytics	Oracle	Agregaciones y endpoints de lectura (consume Kafka)	/api/report/* (read-only)

Pantallas propuestas (ejemplos)

Login

- Ruta: /login
- Componente: LoginComponent
- Acceso: público (no autenticado)
- Descripción:
    - Implementa MSAL para iniciar sesión con Azure AD.
    - Muestra botón "Iniciar sesión con Microsoft".
    - Redirige automáticamente tras el redirectUri configurado.
    - Componente auxiliar: /auth/callback — MSAL redirige aquí tras login exitoso.

Dashboard principal (Home)

- Ruta: /dashboard
- Roles: todos los usuarios autenticados
- Component: DashboardComponent
- Función:
    - Vista inicial tras login (layout base, menú lateral y header).
    - Resumen de actividad según rol:
    - Admin: KPIs globales (pedidos, ventas, usuarios activos).
    - Operator: pedidos en curso y pendientes.
    - Customer: últimos pedidos y estado actual.

Gestión de pedidos (CRUD / seguimiento)

- Ruta: /orders
- Roles: Admin, Operator, Customer (distintos niveles de acceso)
- Component: OrdersComponent
- Subcomponentes:
    - OrderListComponent: tabla de pedidos filtrable.
    - OrderDetailComponent: ver/editar un pedido específico.
    - OrderStatusBadgeComponent: muestra estados visualmente (CREATED, ACCEPTED, etc.).

- Funciones:
    - Listar pedidos (según rol).
    - Crear nuevos pedidos (solo cliente u operador).
    - Cambiar estado del pedido (operador/admin).
    - Llamadas seguras al backend (JWT -> API Gateway -> ms-pedidos360-bff -> ms-pedidos360-orders).

Catálogo de productos

- Ruta: /catalog
- Roles: Admin, Operator
- Component: CatalogComponent
- Subcomponentes:
    - ProductCardComponent
    - ProductFormComponent
- Funciones:
    - Mostrar productos (grid o lista).
    - Crear/editar productos.
    - Control de stock.

Reportería y KPIs

- Ruta: /reports
- Roles: Admin
- Component: ReportsComponent
- Subcomponentes:
    - SalesChartComponent
    - LeadTimeChartComponent
    - TopProductsChartComponent
- Funciones:
    - Consultar KPIs en tiempo real (consume ms-pedidos360-bff -> ms-pedidos360-report -> Kafka clúster).

Auditoría (timeline de eventos)

- Ruta: /audit
- Roles: Admin
- Component: AuditComponent
- Funciones:
    - Mostrar la trazabilidad completa de pedidos o acciones de usuarios.
    - Llamadas a ms-pedidos360-audit (eventos almacenados de Kafka).
    - Permite filtros: usuario, rango de fechas, tipo de evento.

Despliegue (EC2 + Docker/Docker-Compose)

Instancias EC2:

- ec2-apps: orders-svc, catalog-svc, notify-svc, report-svc, audit-svc
- ec2-mq: rabbitmq (opcionalmente 3 nodos) con management UI
- ec2-kafka: zookeeper + kafka (1 broker para MVP, luego escalar)

Docker-Compose por VM (un compose.yml para apps, otro para mq, otro para kafka).
Security Groups: abrir solo puertos necesarios (AMQP/5672, Kafka/9092, HTTP APIs internas).

Integración con API Gateway (protección por JWT Azure)

- Crear HTTP API con JWT Authorizer (issuer/audience de Azure AD).
- Configurar routes → integrations.
- Headers: el Front envía Authorization: Bearer <token> (MSAL).

Requisitos técnicos obligatorios

- Angular 18+ con MSAL Angular y guards por rol.
- Spring Boot 3+, Java 21, OpenAPI por servicio (Swagger).
- AWS API Gateway HTTP API con JWT authorizer (Azure AD).
- EC2 + Docker/Docker-Compose (apps, RabbitMQ, Kafka+ZK).
- RabbitMQ para commands (notificaciones).
- Kafka para events (analítica/auditoría).

Sugerencias de estructura de repositorios GitHub

- /frontend-pedidos360 (Angular + MSAL)
- /ms-pedidos360-bff (Spring Boot, Spring Security)
- /ms-pedidos360-orders (Spring Boot)
- /ms-pedidos360-catalog (Spring Boot)
- /ms-pedidos360-notify (Spring Boot, consumer RabbitMQ)
- /ms-pedidos360-report (Spring Boot, consumer Kafka)
- /ms-pedidos360-audit (Spring Boot, consumer Kafka)
- /infra
    - /apps/compose.yml
    - /mq/compose.yml RabbitMQ)
    - /kafka/compose.yml Zookeeper + Kafka)
    - /docs/ (cualquier documentación técnica solicitada)

Topología RabbitMQ (ejemplo)

6 colas en total, tres flujos de trabajo, cada uno con su cola principal y su DLQ.

Cola principal	Propósito	DLQ asociada
q.cmd.email	Envío de correos/push	q.cmd.email.dlq
q.cmd.kitchen	Ticket/impresión cocina	q.cmd.kitchen.dlq
q.cmd.invoice	Generación de PDF/boleta/factura	q.cmd.invoice.dlq

Exchanges

- cmd.direct (direct): enrutamiento exacto por routing key.
- cmd.topic (topic): enrutamiento por patrones (* = 1 palabra, # = 0..n palabras) para variantes (prioridades o formatos, etc.).
- cmd.dead.dlx (direct): exchange de DLQ.

Bindings (direct + topic)

Exchange	Queue	Binding key / Pattern
cmd.direct	q.cmd.email	email.send

cmd.direct	q.cmd.kitchen	kitchen.ticket
cmd.direct	q.cmd.invoice	invoice.gen
cmd.topic	q.cmd.email	email.* *(ej.: email.send, email.send.high)*
cmd.topic	q.cmd.kitchen	kitchen.# *(ej.: kitchen.ticket, kitchen.ticket.thermal)*
cmd.topic	q.cmd.invoice	invoice.* *(ej.: invoice.gen, invoice.gen.pdf)*
cmd.dead.dlx	q.cmd.email.dlq	email.send
cmd.dead.dlx	q.cmd.kitchen.dlq	kitchen.ticket
cmd.dead.dlx	q.cmd.invoice.dlq	invoice.gen

Buenas prácticas a considerar:

- Definir un envelope común (type, eventId, timestamp, traceId, correlationId) para todas las tareas.
- Realizar ACK/NACK explícitos.
- Idempotencia en comandos (evita efectos secundarios duplicados).
- Métricas/alertas para tasa de DLQ por cola.

Topología Kafka (ejemplo)

Tópico	Particiones	Política	Retención
orders.events	3	delete	3–7 días
audit.timeline	3	compact,delete	14–30 días
*.DLT (por consumidor)	3	delete	7–14 días

orders.events: flujo canónico de eventos de negocio

- Propósito: publicar todo cambio relevante de un pedido (y eventos asociados del dominio). Es la “fuente de verdad” para reportería y auditoría.
- Eventos típicos (mínimos): OrderCreated, OrderAccepted, OrderPreparing, OrderDispatched, OrderDelivered, OrderCancelled

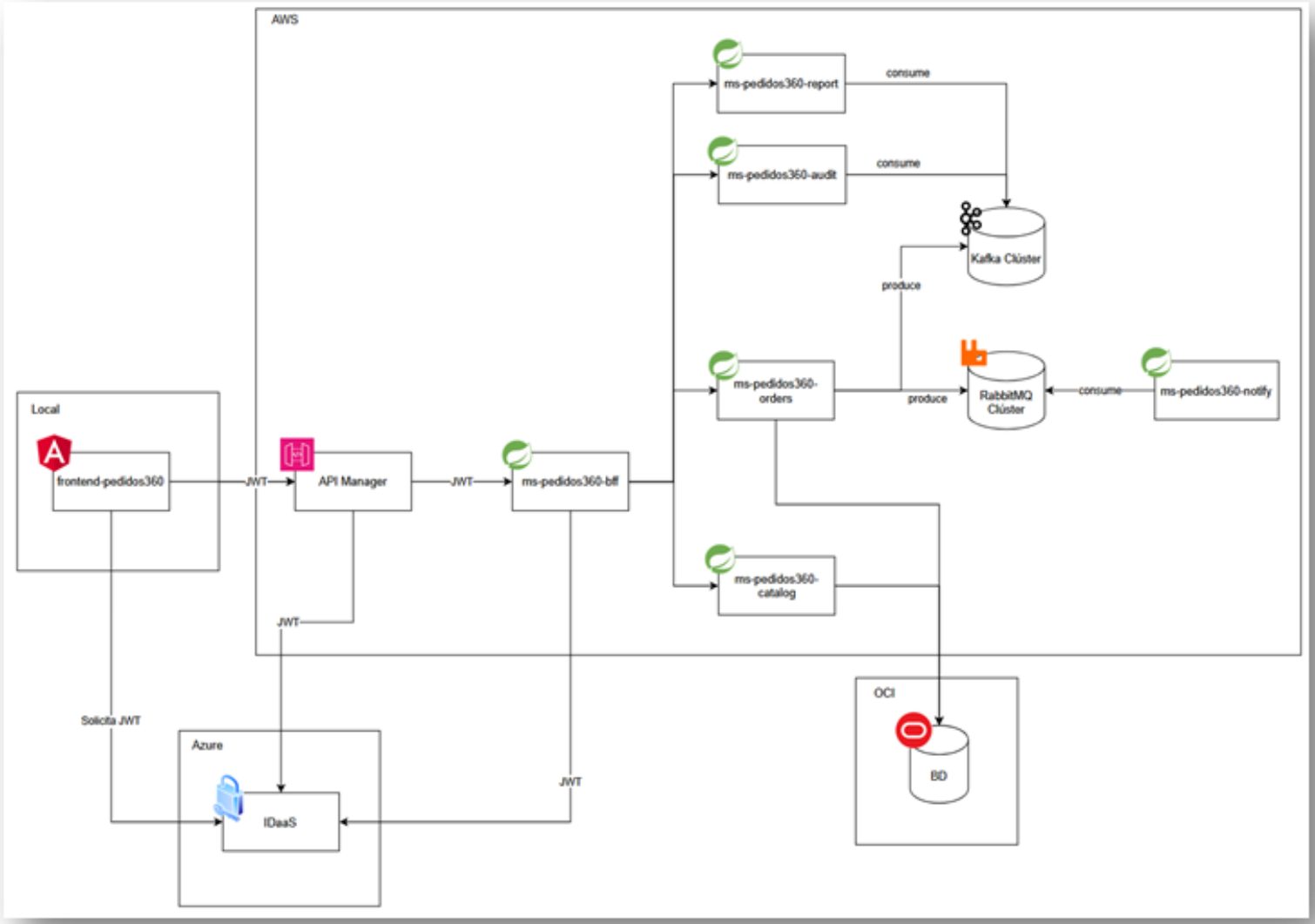
audit.timeline: línea de tiempo/auditoría por entidad

- Propósito: conservar un historial legible y consultable por pedido/actor con foco en cumplimiento y trazabilidad.
- Registro con “quién / qué / cuándo / desde dónde”.

*.DLT: Dead Letter Topics por consumidor

- Propósito: capturar mensajes que no pudieron procesarse tras N reintentos en un consumidor específico (Spring for Apache Kafka).
- Almacenar mensaje original + metadatos de error (stack, timestamp, consumer, offset, etc).

Arquitectura sugerida (ejemplo)


AWS
ms-pedidos360-report
consume
ms-pedidos360-audit
consume
Kafka Clúster
produce
ms-pedidos360-orders
produce
RabbitMQ Clúster
consume
ms-pedidos360-notify
Local
frontend-pedidos360
JWT
API Manager
JWT
ms-pedidos360-bff
ms-pedidos360-catalog
JWT
Solicita JWT
Azure
IDaaS
JWT
OCI
BD